LithosAnanke Kernel and Capsule System

Volume II of the StarshipOS Technical Reference

Robert A. James

Contents

Chapter 1

LithosAnanke Kernel Architecture

1.1 Design Philosophy

1.2 Boot Sequence

The boot sequence from firmware to FORTH REPL:

1. UEFI Firmware → `uefi_loader.c` (`BOOTX64.EFI`)
2. `ExitBootServices()` — kernel owns hardware; receives `BootInfo{memory map, ACPI, framebuffer}`
3. `kernel_main()` — M1 through M7 milestone initialization:

   M1 Console init (UART 16550 + framebuffer)
   M2 PMM — physical memory manager, bitmap allocator
   M3 VMM — 4-level x86_64 paging
   M4 IDT + APIC interrupts
   M5 TSC + HPET + APIC timer (100 Hz heartbeat)
   M6 `kmalloc` heap (16 MB)
   M7 StarForth VM bootstrap + capsule loading → `ok` REPL

1.3 Milestone Status

Table 1.1: LithosAnanke v1.0.8 milestone status

Milestone	Description	Status
M0–M5	Boot, console, PMM, VMM, IDT/APIC, timer	Complete
M6	`kmalloc` infrastructure	Present (full validation deferred)
M7	StarForth VM bootstrap + capsule execution pipeline	In progress

1.4 Memory Layout

1.5 Source Tree

1.6 Multi-Architecture Support

Table 1.2: Kernel memory layout

Region	Description
Kernel heap	16 MB (`kmalloc`)
Block RAM LBN 0–991	1 MB dedicated RAM blocks
Kernel ramdrive LBN 2048–3071	1 MB for capsule loading
LBN 2048	Entry point for `init.4th`

Listing 1.1: StarKernel build targets

```
make -f Makefile.starkernel              # Build for amd64
make -f Makefile.starkernel ARCH=aarch64
make -f Makefile.starkernel ARCH=riscv64
make -f Makefile.starkernel qemu         # Run in QEMU with OVMF
```

1.7 Experimental Campaign Notes

1.8 StarForth, LithosAnanke, and StarshipOS: A Technical Overview

Author: Robert A. James. Date: 9 April 2026. Status: Working Document.

1.8.1 StarForth

What It Is

StarForth is a pure C99 FORTH-79 virtual machine with no libc dependencies. It is self-adaptive: it observes its own execution behavior and continuously adjusts internal parameters to maintain a conservation invariant called K≡1.0. It runs on x86_64, aarch64, and RISC-V.

Why FORTH

FORTH is correct for this work for reasons beyond familiarity. FORTH words are discrete, classifiable, observable computational units — each word has a definite stack-effect signature, each word consumes a known quantity of computational energy, each word is a particle with measurable properties. This is the foundation of StarForth's self-adaptive behavior.

The Primitive Taxonomy

StarForth implements 23 word modules (18 FORTH-79 standard, 5 StarForth extensions). Every primitive word carries quantum-analog properties derived from its stack-effect signature:

Spin. Derived from net stack-effect arithmetic. A word consuming two values and producing one has spin $-\frac{1}{2}$; these values fall directly from `( n n - n )` notation, as half-integer spin falls from the Dirac equation.

Charge. The net stack delta. Charge conservation across a complete program means the program is stack-balanced.

Heat. Execution frequency and recency. Heat decays over time, analogous to radioactive decay.

Mass. Bytecode footprint of the word definition.

Color. Binding state: white words are fully resolved; colored words carry unresolved forward references.

Three of these quantities — spin, charge, and color — behave as genuine conservation laws across well-formed programs. This is a structural property of the system discovered empirically and subsequently formalized in Isabelle/HOL.

The SSM and James Law

The SSM (Steady State Machine) is the self-adaptive core of StarForth. It monitors execution behavior through instrumented subsystems and continuously adjusts internal parameters to maintain K≡1.0.

James Law ($K \equiv 1.0$) is a conservation invariant: the normalized total execution energy of the StarForth universe is conserved at unity across all configurations and workloads. Validation basis: 38,400 experimental runs spanning 128 factorial parameter configurations, CV below 1%, published on SSRN (abstract 5930134).

The SSM operates through four mechanisms:

Rolling Window of Truth (RWOT). A sliding observation window encoding five observables: word identity, prev→word transition, instruction pointer position, execution heat, and causal stack depth. Updated at heartbeat tick intervals, making it a coarse-grained lattice observable.

Adaptive Heartbeat. A self-adjusting timer governing when the SSM evaluates system state and adjusts parameters. The system's intrinsic rhythm.

Physics Hooks. Every word execution in the inner interpreter loop is bracketed by `physics_pre_execute` and `physics_post_execute` calls, updating heat, decay, and window measurements.

Decay. Execution heat decays between heartbeat ticks, keeping the observable window relevant to current rather than historical behavior.

The Phase Space and Attractor

Six workload types have been studied experimentally: STABLE, VOLATILE, TRANSITION, TEMPORAL, DIVERSE, and OMNI. Each produces a distinct trajectory in phase space. All trajectories converge to the same attractor: K≡1.0. Phase-space portrait visualizations confirm attractor convergence across all workload types on all three target architectures.

The Manifold Navigation Controller (Theoretical)

Given a bounded system with a known conservation invariant and predictable attractor manifold, a controller can:

1. Identify the system's current phase-space position using three observables: RWOT (position), adaptive heartbeat metadata (velocity), and spin-class transition rate (acceleration — requires instrumentation).
2. Compute a perturbance vector sufficient to transport the system from its current manifold to an arbitrary target manifold.
3. Inject the perturbance — the burn — and release.
4. Allow the system to coast back to K≡1.0 under its own restoring force.

The controller fires once and releases. The conservation invariant guarantees the return journey. This orbital-mechanics model scales from the word level through the VM, OS, FPGA, and prospectively to custom silicon.

—

1.8.2 LithosAnanke

What It Is

LithosAnanke is a bare-metal operating system for x86_64, currently at milestone M7 (VM integration). It boots via UEFI, manages physical and virtual memory, handles interrupts via IDT/APIC, maintains a timer, manages a heap, and hosts the StarForth VM as its primary computational substrate. Written in C99 with no external dependencies.

The Capsule System

The primary organizational unit is the *capsule*: a content-addressed, 64-byte cache-line-aligned descriptor. A capsule's ID is its content hash. Identity is content: two capsules with identical content have identical IDs. Modification produces a new capsule with a new ID. Capsules are immutable by design; mutation produces new capsules.

Component Architecture

Hera. The foundational VM and heartbeat. The system's ground state.

Hermes. Message entropy and TTL decay. Manages the message-passing fabric, monitors message lifecycle, and reaps expired messages. The natural home of the perturbance controller extension.

Artemis. Persistence and event sourcing. Manages the capsule store, maintains the event log, and handles durable state.

Components communicate exclusively through message passing. There is no shared mutable state between components.

Authentication and Identity

LithosAnanke implements a hardware-rooted authentication model with no usernames and no passwords. Identity is carried on a physical thumb drive containing two raw block partitions.

Access control is governed by temporal ACL grants that decay via the same Hermes TTL mechanism that governs message lifetime. The default state of the permission system is no access. Decay to zero requires no explicit revocation.

Normalized Virtual Block Space

From any VM's perspective, LithosAnanke presents a single contiguous block address space. Physical origin — local drive, system hardware, or remote cloud mount — is transparent. Blocks appear as grants are issued; blocks vanish as grants decay. Security is expressed as address-space geometry.

The Three UX Tiers

CLI. The traditional FORTH terminal interface. Full power, zero overhead. The REPL is operational.

GUI. A wireframe soundstage rendered from rasterized vector graphics. Words are physical objects; the user assembles colon definitions spatially. The FORTH text is always visible underneath.

VR. The full holodeck realization. The user stands inside the phase space. Words have mass, spin, heat, and charge as observable properties. Manifold navigation is literally navigable space.

All three tiers are windows into identical underlying mechanics.

—

1.8.3 StarshipOS

The SSPU

The SSPU (Steady State Processing Unit) is a novel processor architecture extending the StarForth/SSM work into hardware. It communicates entirely via a message-passing fabric called Gefyra. There are no traditional buses. Targeted for initial FPGA implementation on a Digilent Genesys ZU (Zynq UltraScale+).

Milestone Status

Milestone	Description	Status
M0	UEFI boot	Complete
M1	Physical memory manager	Complete
M2	Virtual memory manager	Complete
M3	IDT/APIC interrupt handling	Complete
M4	Timer	Complete
M5	Heap	Complete
M6	–	Complete
M7	VM integration	Active frontier
–	Manifold Navigation Controller	Theoretical
–	SSPU FPGA implementation	Planned (Genesys ZU)

Design Philosophy

The recurring pattern in this architecture is the elimination of problem domains rather than their simplification. No scheduler, no traditional memory manager, no username/password authentication, no logout ceremony, no coordination layer for distributed state — each elimination is a deliberate architectural decision. The question at each design point was not “how do we implement this?” but “do we need this at all?”

Key Terms

Term	Definition
K≡1.0	James Law — conservation invariant of normalized execution energy
SSM	Steady State Machine — self-adaptive core of StarForth
RWOT	Rolling Window of Truth — coarse-grained phase-space observable
SSPU	Steady State Processing Unit — novel message-passing processor
Gefyra	Message-passing fabric connecting SSPU processing elements
Capsule	Content-addressed, 64-byte aligned immutable descriptor
Hera	Foundational VM and heartbeat component
Hermes	Message entropy, TTL decay, lifecycle management
Artemis	Persistence and event sourcing

Chapter 2

Capsule System

2.1 Capsule Types

Table 2.1: Capsule types

Type code	Name	Purpose
`(m)`	`MAMA_INIT`	Exactly one Mama VM initialization capsule
`(p)`	`PRODUCTION`	Truth-bearing baby VM initializers
`(e)`	`EXPERIMENT`	DoE workload-only initializers

2.2 Birth Protocol

The capsule birth protocol:

1. Locate capsule by name
2. Validate content hash (XXHash64)
3. Allocate VM ID
4. Execute IDENTITY (capsule code)
5. Execute PERSONALITY (block 1 from ramdrive)
6. Log parity record: VM ID + capsule hash + dict hash

2.3 Content Addressing and Immutability

2.4 Parity Logging

2.5 Capsule Files

2.6 Mama Forth Supervisor

Chapter 3

Word-Level ACL and Security

3.1 ACL System in Kernel Context

3.2 Two-Console Model

3.3 Zuse Superuser

3.4 Future: PKI and Ed25519 (Phase 8)

Chapter 4

Platform Support

4.1 Linux / Hosted

4.2 L4Re / Fiasco.OC

4.3 Bare Metal: amd64

4.4 Bare Metal: aarch64 and riscv64

4.5 Platform Abstraction Layer (HAL Target)

4.6 Raspberry Pi Build

Listing 4.1: Raspberry Pi cross-compilation

```
1 make rpi4-cross     # Cross-compile for Raspberry Pi 4 (ARM64)
```

Chapter 5

L8 Jacquard Mode Selector

5.1 Six Operating Modes

Table 5.1: L8 Jacquard operating modes

Mode	Characteristics
`stable`	—
`volatile`	—
`diverse`	—
`temporal`	—
`transition`	—
`omni`	—

5.2 Attractor Bucket Mechanism

5.3 ssm_jacquard.c Implementation

5.4 Jacquard Capsule Variants

Chapter 6

QEMU Testing and Regression Baseline

6.1 Running the Kernel in QEMU

Listing 6.1: Running LithosAnanke in QEMU

```
make -f Makefile.starkernel qemu
# Requires OVMF UEFI firmware in the OVMF_PATH (see Makefile.
    starkernel)
```

6.2 Regression Baseline

The file `QEMU_BASELINE.log` at the repository root captures the reference QEMU/OVMF output from a known-good LithosAnanke build. Any deviation in a new build's output from this baseline indicates a regression.